Applied NLP

DATA 641

Lecture 4 — Text Representation; TF-IDF

*Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems 1st Edition, Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, Harshit Surana, O' Reilly.

TF-IDF

- In all the three approaches we have seen so far, all the words in the text are treated as equally important, there is no notion of some words in the document being more important than others.

- TF-IDF, or term frequency–inverse document frequency, addresses this issue. It aims to quantify the importance of a given word relative to other words in the document and in the corpus.

- The intuition behind TF-IDF is as follows: if a word w appears many times in a document d_i but does not occur much in the rest of the documents d_j in the corpus, then the word w must be of great importance to the document d_i. The importance of w should increase in proportion to its frequency in d_i, but at the same time, its importance should decrease in proportion to the word's frequency in other documents d_j in the corpus. Mathematically, this is captured using two quantities: TF and IDF. The two are then combined to arrive at the **TF-IDF score**.

- TF (*term frequency*) measures how often a term or word occurs in a given document. Since different documents in the corpus may be of different lengths, a term may occur more often in a longer document as compared to a shorter document. To normalize these counts, we divide the number of occurrences by the length of the document. TF of a term t in a document d is defined as

$$TF(t,d) = \frac{\text{Number of occurrences of term } t \text{ in document } d}{\text{Total number of terms in the document } d} \tag{1}$$

- IDF (*inverse document frequency*) measures the importance of the term across a corpus. In computing TF, all terms are given equal importance (weightage). However, it is a well-known fact that stop words like is, are, am, etc., are not important, even though they occur frequently. To account for such cases, IDF weighs down the terms that are very common across a corpus and weighs up the rare terms. IDF of a term t is calculated as follows:

$$IDF(t) = \log_e \frac{\text{Total number of documents in the corpus}}{\text{Number of documents with term } t \text{ in them}} \tag{2}$$

The $TF - IDF$ score is a product of these two terms. Thus, $TF * IDF$.

Note: There are several variations of the basic TF-IDF formula that are used in practice. The TF-IDF scores that we can calculate for our corpus might not match the TF-IDF scores given by scikit-learn.

Some of the **disadvantages** of this encoding include:

- TFIDF similarly to the other representations are discrete representations—i.e., they treat language units (words, n-grams, etc.) as atomic units. This discreteness hampers their ability to capture relationships between words.
- The feature vectors are sparse and high-dimensional representations. The dimensionality increases with the size of the vocabulary, with most values being zero for any vector. This hampers learning capability. Further, high-dimensionality representation makes them computationally inefficient.
- They cannot handle OOV words.